

Employee Management System (Console-based Java Application)

CODE:

```
1 package day_7;
2
3 import java.util.ArrayList;
4 import java.util.Scanner;
5
6 public class EmployeeManagementSample {
7
8     Scanner sc = new Scanner(System.in);
9
10
11     ArrayList<Employee> employees = new ArrayList<>();
12
13
14     static class Employee {
15         String name;
16         int age;
17         String designation;
18         double salary;
19
20         Employee(String name, int age, String designation, double salary) {
21             this.name = name;
22             this.age = age;
23             this.designation = designation;
24             this.salary = salary;
25         }
26     }
27
28     public static void main(String[] args) {
29         EmployeeManagementSample obj = new EmployeeManagementSample();
30         int choice;
31
32         do {
33             System.out.println("\n----- MENU -----");
34             System.out.println("1) Create");
35             System.out.println("2) Display");
36             System.out.println("3) Raise Salary");
37             System.out.println("4) Exit");
38             System.out.print("Enter your choice: ");
39             choice = obj.sc.nextInt();
40
41             switch (choice) {
42                 case 1:
43                     obj.create();
44                     break;
45                 case 2:
46                     obj.display();
47                     break;
48                 case 3:
49                     obj.raiseSalary();
50                     break;
51                 case 4:
52                     System.out.println("Exiting program... Goodbye!");
53                     break;
54                 default:
55                     System.out.println("Invalid choice! Please enter 1-4.");
56             }
57         } while (choice != 4);
58     }
59
60     void create() {
61         boolean addMore = true;
62
63         while (addMore) {
64             sc.nextLine();
65             System.out.print("Enter the name: ");
66             String name = sc.nextLine();
67
68             int age;
69             do {
70                 System.out.print("Enter the age (must be above 18): ");
71                 age = sc.nextInt();
72                 if (age <= 18) {
73                     System.out.println("Invalid age! Age must be above 18.");
74                 }
75             } while (age <= 18);
76
77             sc.nextLine();
78             System.out.print("Enter the designation (P - Programmer, T - Tester, M - Manager): ");
79             String code = sc.nextLine().toUpperCase();
80
81             Employee emp = new Employee(name, age, code, 0.0);
82             employees.add(emp);
83
84             System.out.print("Do you want to add more? (yes/no): ");
85             String ans = sc.nextLine();
86             addMore = ans.equalsIgnoreCase("yes");
87         }
88     }
89
90     void display() {
91         if (employees.isEmpty()) {
92             System.out.println("No employees found.");
93         } else {
94             System.out.println("List of Employees:");
95             for (Employee emp : employees) {
96                 System.out.println(emp.name + " | " + emp.age + " | " + emp.designation + " | " + emp.salary);
97             }
98         }
99     }
100
101     void raiseSalary() {
102         if (employees.isEmpty()) {
103             System.out.println("No employees found.");
104         } else {
105             System.out.println("Enter the index of the employee whose salary you want to raise:");
106             for (int i = 0; i < employees.size(); i++) {
107                 System.out.print(i + " | ");
108                 if (i % 5 == 4) System.out.println();
109             }
110
111             int index = -1;
112             do {
113                 index = -1;
114                 System.out.print("Enter the index: ");
115                 int idx = sc.nextInt();
116                 if (idx < 0 || idx >= employees.size()) {
117                     System.out.println("Invalid index. Please enter a valid index between 0 and " + (employees.size() - 1));
118                 } else {
119                     index = idx;
120                 }
121             } while (index == -1);
122
123             Employee emp = employees.get(index);
124             System.out.print("Enter the new salary: ");
125             double newSalary = sc.nextDouble();
126             emp.salary = newSalary;
127             System.out.println("Salary of " + emp.name + " has been updated to " + emp.salary);
128         }
129     }
130 }
```

```
81     String designation;
82     double salary;
83
84     switch (code) {
85         case "P":
86             designation = "Programmer";
87             salary = 35000;
88             break;
89         case "T":
90             designation = "Tester";
91             salary = 25000;
92             break;
93         case "M":
94             designation = "Manager";
95             salary = 50000;
96             break;
97         default:
98             designation = "Not Specified";
99             salary = 0;
100     }
101
102     employees.add(new Employee(name, age, designation, salary));
103
104     System.out.println("Employee created successfully! Base salary set to " + salary);
105
106     String response;
107     while (true) {
108         System.out.print("Do you want to add another employee? (Y/N): ");
109         response = sc.nextLine().trim().toUpperCase();
110         if (response.equals("Y") || response.equals("N")) {
111             break;
112         }
113         System.out.println("Invalid input! Please enter Y or N.");
114     }
115
116     addMore = response.equals("Y");
117 }
118
119 }
120 }
```

EmployeeManagementSample (1) [Java]

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice:

Javadoc Declaration Debug

day_7

```
122 void display() {
123     if (employees.isEmpty()) {
124         System.out.println("No employee records found. Please create one first.");
125         return;
126     }
127
128     System.out.println("\n--- Employee Details ---");
129     for (int i = 0; i < employees.size(); i++) {
130         Employee e = employees.get(i);
131         System.out.println("\nEmployee #" + (i + 1));
132         System.out.println("Name: " + e.name);
133         System.out.println("Age: " + e.age);
134         System.out.println("Designation: " + e.designation);
135         System.out.println("Salary: " + e.salary);
136     }
137 }
138
139 void raiseSalary() {
140     if (employees.isEmpty()) {
141         System.out.println("No employee records found. Please create one first.");
142         return;
143     }
144
145     sc.nextLine();
146     System.out.print("Enter the name of the employee to raise salary for: ");
147     String searchName = sc.nextLine();
148
149     Employee target = null;
150     for (Employee e : employees) {
151         if (e.name.equalsIgnoreCase(searchName)) {
152             target = e;
153             break;
154         }
155     }
156
157     if (target == null) {
158         System.out.println("No employee found with the name: " + searchName);
159         return;
160     }
161 }
```

EmployeeManagementSample

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice:

Javadoc Declaration Debug

day_7

```

140     if (employees.isEmpty()) {
141         System.out.println("No employee records found. Please create one first.");
142         return;
143     }
144
145     sc.nextLine();
146     System.out.print("Enter the name of the employee to raise salary for: ");
147     String searchName = sc.nextLine();
148
149
150     Employee target = null;
151     for (Employee e : employees) {
152         if (e.name.equalsIgnoreCase(searchName)) {
153             target = e;
154             break;
155         }
156     }
157
158     if (target == null) {
159         System.out.println("No employee found with the name: " + searchName);
160         return;
161     }
162
163
164     double percent;
165     do {
166         System.out.print("Enter the raise percentage (must be between 1 and 10): ");
167         percent = sc.nextDouble();
168         if (percent < 1 || percent > 10) {
169             System.out.println("Invalid percentage! It must be between 1 and 10.");
170         }
171     } while (percent < 1 || percent > 10);
172
173     double increment = target.salary * (percent / 100);
174     target.salary = target.salary + increment;
175
176     System.out.println("Salary raised successfully for " + target.name + "!");
177     System.out.println("New Salary: " + target.salary);
178 }
179 }

```

EmployeeManagementSample (1)

```

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice:

```

Javadoc X Declaration Debug

day_7

OUTPUT:

```
Console X
<terminated> EmployeeManagementSample (1) [Java Application] C:\Users\Vinay Kumar B\Downloads\spring-tools-for-eclipse-5.3.0.RELEASE-e4.40.0-win32.win32.x86_64\sts-5.3.0.RELEASE

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice: 1
Enter the name: Vinay
Enter the age (must be above 18): 20
Enter the designation (P - Programmer, T - Tester, M - Manager): P
Employee created successfully! Base salary set to 35000.0
Do you want to add another employee? (Y/N): Y
1
Enter the name: Kumar
Enter the age (must be above 18): 20
Enter the designation (P - Programmer, T - Tester, M - Manager): M
Employee created successfully! Base salary set to 50000.0
Do you want to add another employee? (Y/N): N

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice: 2

--- Employee Details ---

Employee #1
Name: Vinay
Age: 20
Designation: Programmer
Salary: 35000.0

Employee #2
Name: Kumar
Age: 20
Designation: Manager
Salary: 50000.0
```

```
Console X
<terminated> EmployeeManagementSample (1) [Java Application] C:\Users\Vinay Kumar B\Downloads\spring-tools-for-eclipse-5.3.0.RELEASE-e4.40.0-win32.win32.x86_64\sts-5.3.0.RELEASE

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice: 3
Enter the name of the employee to raise salary for: Vinay
Enter the raise percentage (must be between 1 and 10): 4
Salary raised successfully for Vinay!
New Salary: 36400.0

----- MENU -----
1) Create
2) Display
3) Raise Salary
4) Exit
Enter your choice: 2

--- Employee Details ---

Employee #1
Name: Vinay
Age: 20
Designation: Programmer
Salary: 36400.0

Employee #2
Name: Kumar
Age: 20
Designation: Manager
Salary: 50000.0
```

```
----- MENU -----  
1) Create  
2) Display  
3) Raise Salary  
4) Exit  
Enter your choice: 4  
Exiting program... Goodbye!
```

Declaration  Debug



What it does:

This is a simple menu-driven Java console application that lets a user manage employee records in memory using an ArrayList. It demonstrates core Java concepts like OOP, collections, loops, switch-case, and input validation.

Key Features:

1. Create Employee :

- Takes employee name, age (validated to be above 18), and designation code (P/T/M).
- Automatically assigns designation and base salary based on the code:
 - P → Programmer, ₹35,000
 - T → Tester, ₹25,000
 - M → Manager, ₹50,000
 - Anything else → "Not Specified", ₹0
- Allows adding multiple employees in a loop (asks Y/N after each entry).

2. Display Employees :

- Lists all stored employees with their name, age, designation, and salary.
- Shows a message if no records exist.

3. Raise Salary :

- Searches for an employee by name (case-insensitive).
- Validates and applies a percentage-based salary raise (must be between 1% and 10%).
- Updates and displays the new salary.
-

4. Exit :

- Ends the program gracefully.